hashlock.

Security Audit

Max (DeFi)

July 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

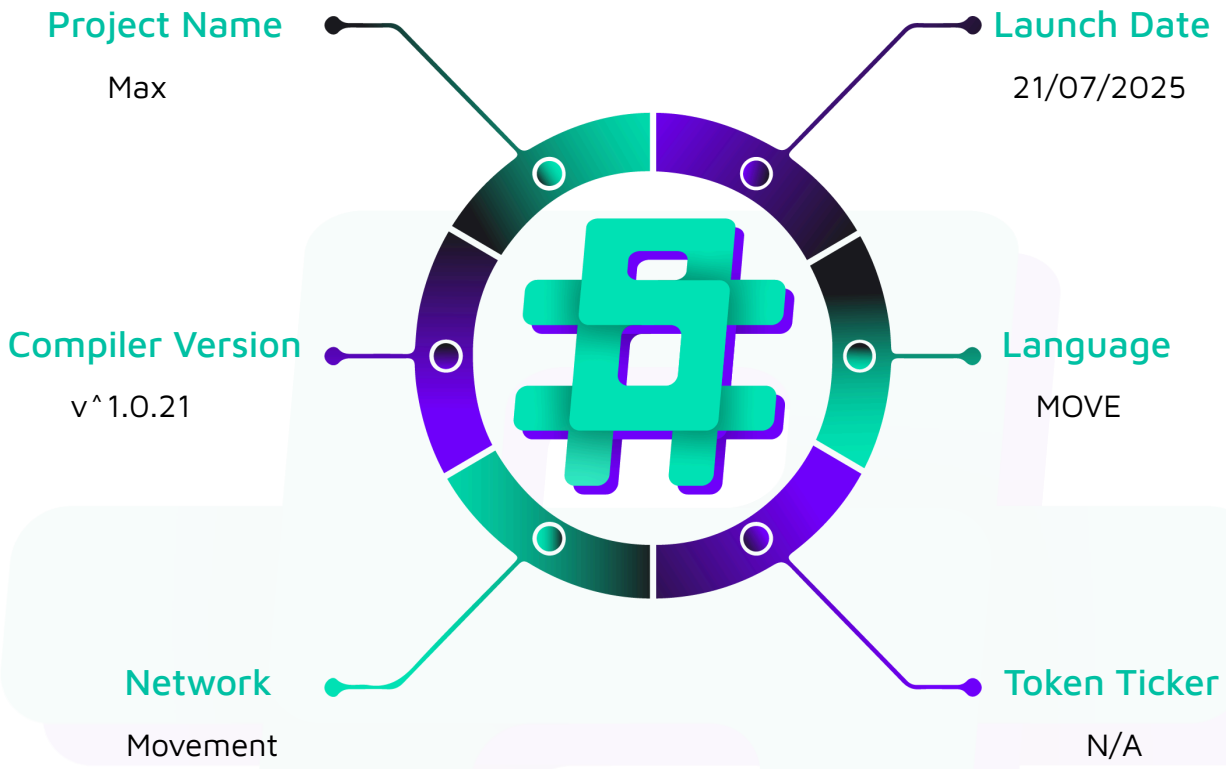
The Max team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

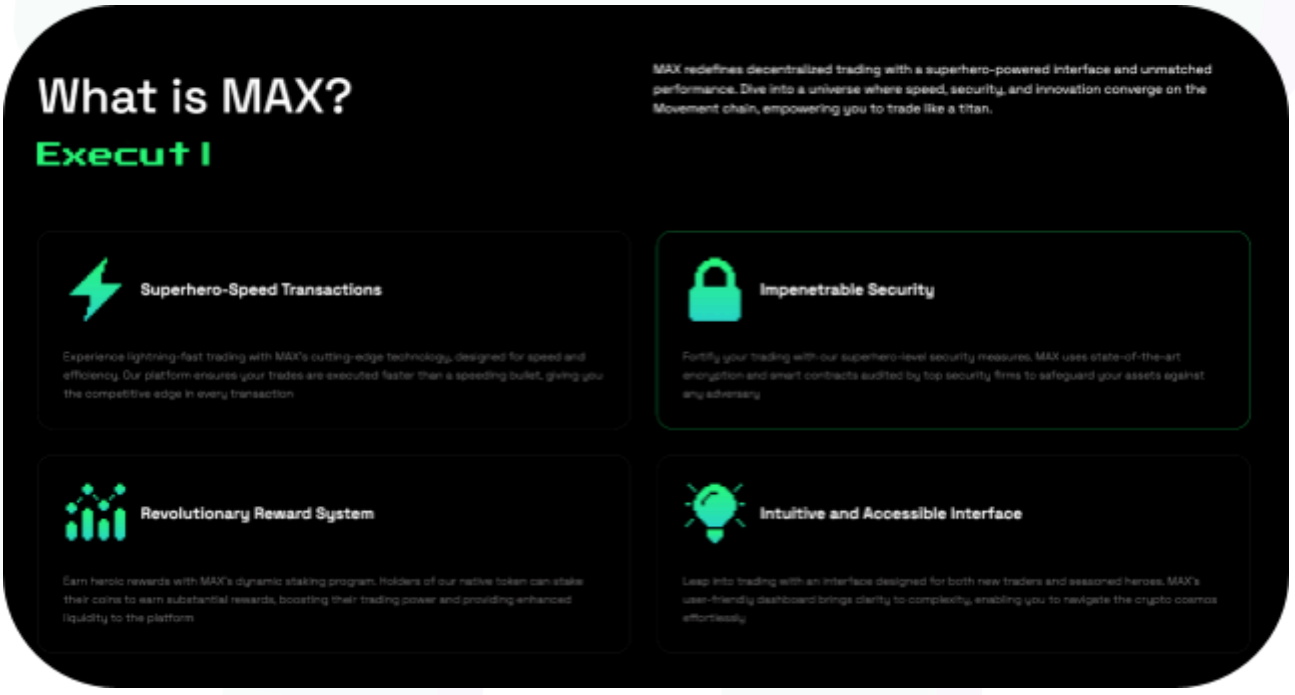
MAX is a high-performance decentralized exchange (DEX) protocol built on the Movement Labs blockchain. Designed for speed, scalability, and precision, it enables capital-efficient trading, dynamic liquidity management, and AI-powered execution through a modular smart contract architecture.

Project Name: Max
Project Type: DeFi, L2
Compiler Version: v^1.0.21
Website: https://www.max.mov/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the move code within the Max project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Max Smart Contracts**
Platform	**Aptos / Move**
Audit Date	**July, 2025**
Components	- libs/* - Config.move - Emergency.move - Fee_tier.move - Liquidity_pool.move - Position_nft_manager.move - Router.move - scripts.move
GitHub Repo	https://github.com/max-mov-dex/max-contracts
GitHub Commit Hash	9a4eb9e8dcc89cda106440be28a195ee91c1200e
Fix Review GitHub Commit Hash	4398a9523e1a1ed7b22c77b08e8431b4530b6ab4

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 Medium severity vulnerabilities

7 Low severity vulnerabilities

1 Gas Optimisation

4 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
libs/* - Libraries that are used for low-level arithmetic and swap computations.	**Contract achieves this functionality.**
config.move Allows the admins to: - Update the pool admin, reward admin, and emergency admin addresses. Allows the pool admin to: - Update protocol fees. - Update trader fees.	**Contract achieves this functionality.**
emergency.move Allows the emergency admin to: - Pause the protocol. - Resume the protocol. - Disable the protocol.	**Contract achieves this functionality.**
fee_tier.move Allows the pool admin to: - Add fee tiers. - Delete fee tiers.	**Contract achieves this functionality.**
liquidity_pool.move Allows users to: - Create pools. - Open positions. - Close positions. - Add liquidity into pools. - Remove liquidity from pools.	**Contract achieves this functionality.**

- Swap tokens. - Collect accrued fees. - Collect rewards. - Interact with oracle module. Allows the pool admin to: - Collect protocol fees. Allows the reward manager to: - Update the reward manager address for a reward. - Update reward emissions. - Add rewards. - Remove rewards. Allows the reward admin to: - Create new rewards.	
position_nft_manager.move Allows users to: - Create NFT collection. - Mint positions. - Burn positions. - Helper functions that interact with liquidity_pool.move.	**Contract achieves this functionality.**
router.move, scripts.move - Helper functions that interact with liquidity_pool.move. - Supports creating a pool and adding liquidity with multiple coins/fungible token assets. - Supports swapping between coins and FTA	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Max project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Max project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] sources/config.move#emergency_admin - Incorrect admin address returned in view function

Description

The `emergency_admin` view function in line 146 is intended to return the emergency admin's address. However, it incorrectly returns the pool admin's address.

Impact

The `emergency_admin` view function returns an incorrect address, which may cause unintended consequences for its caller or third-party modules.

Recommendation

Consider updating the function to return the emergency admin's address.

Status

Resolved

[M-02] sources/liquidity_pool.move - Emergency status is not enforced sufficiently

Description

Across the contract, the `emergency::assert_no_emergency()` function is called to prevent users from interacting with the pool when the emergency admin halts the protocol. It is defined in `sources/emergency.move`, lines 63-65.

However, this restriction is not enforced in the following entry points:

- `collect_protocol_fee`
- `update_reward_manager`
- `update_reward_emissions`
- `add_reward`
- `remove_reward`
- `collect_reward`
- `increase_observation_cardinality_next`
- `observe`
- `initialize_reward`

Impact

If the emergency admin halts the protocol due to unforeseen bugs or unintended exploits, the above functions can still be called, which may lead to a loss of funds or incorrect state consequences.

Recommendation

Consider applying the `emergency::assert_no_emergency()` validation in the above entry points.

Status

Resolved

[M-03] sources/liquidity_pool.move#assert_ticks - tick_lower is not checked with tick::min_tick()

Description

The `assert_ticks` function in line 1564 performs validation for the `tick_lower` and `tick_upper` parameters. However, it does not ensure that the `tick_lower` is greater than or equal to `tick::min_tick()`.

Impact

Users may open positions with invalid `tick_lower` values.

Recommendation

Consider adding validation to ensure `tick_lower` is greater than or equal to `tick::min_tick()`.

Status

Resolved

Low

[L-01] sources/config.move - Two-step ownership transfer is not implemented

Description

The `set_pool_admin`, `set_reward_admin`, and `set_emergency_admin` functions in lines 67-83 allow the privileged addresses to be updated in a transaction. This is problematic because if the ownership is set to an incorrect address, the ownership will be lost.

Recommendation

Consider implementing a two-step ownership transfer process: the first transaction involves nominating a new address, and the second transaction requires the nominated address to accept the ownership transfer.

Status

Resolved

[L-02] sources/fee_tier.move#add_fee_tier - AddFeeTierEvent is defined but not emitted

Description

The `AddFeeTierEvent`, defined in line 31, is an event that should be emitted when a new fee tier is added. However, the `add_fee_tier` function in line 55 does not emit the `AddFeeTierEvent`.

Recommendation

Consider emitting the `AddFeeTierEvent` in the `add_fee_tier` function.

Status

Resolved

[L-03] sources/liquidity_pool.move#pay_swap - pay_swap does not emit events

Description

The `pay_swap` function in line 745 is intended for users to deposit funds into the pool during a swap operation. However, no events are emitted when the user pays the swap amount.

Recommendation

Consider emitting an event in the `pay_swap` function.

Status

Acknowledged

[L-04] sources/libs/math_u256.move - E_DIV_BY_ZERO error is not applied fully

Description

The `E_DIV_BY_ZERO` error code in line 6 is defined for division-by-zero errors. However, this error is not applied to instances where division by zero or modulo by zero is attempted.

These instances are illustrated below:

- An error should be triggered if `b` is zero in the `div_mod` and `div_rounding_up` functions.
- An error should be triggered if `c` is zero in the `mul_mod` functions.

Recommendation

Consider adding error validation in the above functions.

Status

Resolved

[L-05] emergency.move, fee_tier.move - Admin permission checks performed after business logic in emergency functions

Description

In multiple modules (`emergency.move` and `fee_tier.move`), admin permission checks are performed after other validation logic, which could lead to unnecessary gas consumption for unauthorized users and missing event emissions.

Vulnerability Details

In the affected functions, the admin permission checks (config::assert_emergency_admin or config::assert_pool_admin) are executed after performing other business logic validations. This means unauthorized users will consume gas for these validations before the transaction is reverted due to permission denial.

Impact

Unnecessary gas consumption for unauthorized users

Recommendation

Move the admin permission checks to the beginning of the functions to fail fast and avoid unnecessary gas consumption:

Status

Resolved

[L-06] Liquidity_pool.move - Unused error code declaration in liquidity_pool.move

Description

There's an unused error code declaration (`E_NOT_ENOUGH_REWARD`) in the `liquidity_pool.move` module, which adds unnecessary code and could lead to confusion for developers or auditors reviewing the codebase.

Vulnerability Details

In the `liquidity_pool.move` file, there's an error code declaration that is never used in the module's implementation

Impact

This vulnerability may mislead developers to believe there's error handling for reward insufficiency when there really is not.

Recommendation

Implement proper error handling using this error code in relevant functions where reward insufficiency might occur

Status

Resolved

[L-07] TickInfo - Typo in Core Liquidity Field of TickInfo Struct

Description

In the `LiquidityPool` module, there's a significant spelling error in the TickInfo struct's field name

Vulnerability Details

The field `liquditiy_gross` is misspelled (should be `liquidity_gross`). While the code still functions since the misspelling is consistent throughout the codebase

Impact

The typo in a core field name reduces code readability and may cause confusion for developers during maintenance or when integrating with the protocol

Recommendation

Rename the field from `liquditiy_gross` to `liquidity_gross` throughout the codebase.

Status

Resolved

Gas

[G-01] sources/liquidity_pool.move#update_position_rewards - Global variables can be declared outside loop

Description

The `update_position_rewards` function in line 1524 iterates over the `reward_growths_inside` vector while declaring the `liquidity` variable inside the for loop. This approach is unnecessary, as the `position.liquidity` variable will not be changed across the iterations.

Recommendation

Consider declaring the `liquidity` variable outside the for loop.

Status

Resolved

QA

[Q-01] sources/config.move - Improve consistency in numbers spacing

Description

The `TRADER_FEE_MULTIPLIER_SCALE` constant in line 21 is defined as `100_00`. This is inconsistent compared to other constant spacing, such as `FEE_SCALE` with a value of `1_000_000`.

Recommendation

Consider updating it to `10_000`.

Status

Resolved

[Q-02] sources/liquidity_pool.move#swap - Repititive code inside if-else statement

Description

The `pool_data.current_sqrt_price = current_sqrt_price` is placed within the if-else statement in lines 679 and 681.

This can be simplified by always calling `pool_data.current_sqrt_price = current_sqrt_price` outside of the statement to avoid repetitive code.

Additionally, the if-else statement can be simplified to a single if statement, removing the else branch.

Recommendation

Consider updating the code as above.

Status

Resolved

[Q-03] sources/liquidity_pool.move#add_delta_liquidity - Hardcoded error code

Description

The `add_delta_liquidity` function in line 1580 calls the `assert!(liqudity_after <= MAX_U128, 1)` to ensure the liquidity amount is less than or equal to the `MAX_U128` value. However, the `1` error code does not indicate any useful information regarding the error.

Recommendation

Consider declaring a meaningful constant name and using it instead.

Status

Resolved

[Q-04] sources/liquidity_pool.move#position_with_pending_fees_and_rewards - Unneeded parsing for Position

Description

The `position_with_pending_fees_and_rewards` function in line 1953 declares the `position_view` variable as a `Position` struct by parsing the fields from the `position` variable.

However, since the `position` and `position_view` variables both use the same type, the parsing of fields is unnecessary.

Recommendation

Consider setting the `position_view` variable to the `position` variable directly.

Status

Resolved

Centralisation

The Max project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Max project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au